Assignment - 1

Aim:- To Implement insertion & preorder traversal operations in a BST.

Algorithm:-

1- Search and add node in a BST

steps:

1. Start from root of the BST
2. If root is NULL, create new node & insert the value.
3. If new value equals the value of the root, the value already exists.
4. If newvalue > rootvalue
   recursively search in Right Subtree.
5. else
   Recursively search in left subtree
6. Insert the newNode & Return.

2. Search maximum Value in BST

steps:

1. Start from root the Node
2. If root is NULL, tree is empty return
3. Move the right child as long as it exists.
4. The rightmost child contains maximum value
   ∴ return the rightest child in the end leaf node

3. Preorder traversal with Recursion steps:

i. If root is Null, return.
ii. Visit & print the root Node.
iii. Recursively traverse the right subtree.

4. Preorder traversal without Recursion steps:

i. if root is null, return.
ii. Initialise an empty stack & push root onto it.
iii. While stack is not empty.
- pop the top node & print it.
- push right child if exists.
- push left child if exists.

5. LevelOrder(root)
(i) enqueue rootNode.
(ii) while (queue is not empty)
- deque all elements of queue & print them.
- and enqueue their non-null children
- print level number and new line.

properties:- BST is a non-linear data structure with each node having between [0, 2] nodes as children (Left, Right) of parent node. In BST the values are in order of left child < parent < right child.

Conclusion:- Successfully implemented Binary Search Tree for Integer data values and printing of traversal preorders.

```c
#include <stdio.h>
#include <stdlib.h>
#include <limits.h>

// --- Structure Definitions ---

typedef struct Node {
    int val;
    struct Node* left;
    struct Node* right;
} Node;

typedef struct Stack {
    Node** data;
    int top;
    int capacity;
} Stack;

typedef struct Queue {
    Node** data;
    int front;
    int rear;
    int capacity;
} Queue;

// --- Node Helper Functions ---

Node* create_node(int val) {
    Node* new_node = (Node*)malloc(sizeof(Node));
    if (!new_node) return NULL;
    new_node->val = val;
    new_node->left = NULL;
    new_node->right = NULL;
    return new_node;
}

void free_tree(Node* root) {
    if (root == NULL) return;
    free_tree(root->left);
    free_tree(root->right);
    free(root);
}

// --- Tree Operations ---

void add(int data, Node** root) {
    if (*root == NULL) {
        *root = create_node(data);
        return;
    }

    Node* temp = *root;
    while (1) {
        if (data < temp->val) {
```

```
52      Node* temp = *root;
53      while (1) {
54          if (data < temp->val) {
55              if (temp->left == NULL) {
56                  temp->left = create_node(data);
57                  return;
58              }
59              temp = temp->left;
60          } else if (data > temp->val) {
61              if (temp->right == NULL) {
62                  temp->right = create_node(data);
63                  return;
64              }
65              temp = temp->right;
66          } else {
67              return; // Duplicate value
68          }
69      }
70  }
71
72  Node* search(Node* root, int data) {
73      Node* temp = root;
74      while (temp != NULL) {
75          if (data < temp->val)
76              temp = temp->left;
77          else if (data > temp->val)
78              temp = temp->right;
79          else
80              return temp;
81      }
82      return NULL;
83  }
84
85  int maxi(Node* root) {
86      if (root == NULL)
87          return INT_MIN;
88
89      while (root->right != NULL)
90          root = root->right;
91
92      return root->val;
93  }
94
95  int mini(Node* root) {
96      if (root == NULL)
97          return INT_MAX;
98
99      while (root->left != NULL)
100         root = root->left;
101
102     return root->val;
103 }
104
105 static inline int max_int(int a, int b) {
```

```c
static inline int max_int(int a, int b) {
    return (a > b) ? a : b;
}

int height(Node* root) {
    if (root == NULL)
        return 0;

    return 1 + max_int(height(root->left), height(root->right));
}

// --- Recursive Traversals ---

void inorder(Node* root) {
    if (root == NULL)
        return;

    inorder(root->left);
    printf("%d ", root->val);
    inorder(root->right);
}

void preorderRec(Node* root) {
    if (root == NULL)
        return;

    printf("%d ", root->val);
    preorderRec(root->left);
    preorderRec(root->right);
}

void postorder(Node* root) {
    if (root == NULL)
        return;

    postorder(root->left);
    postorder(root->right);
    printf("%d ", root->val);
}

// --- Stack Utilities & Iterative Preorder Traversal ---

Stack* create_stack(int capacity) {
    Stack* st = (Stack*)malloc(sizeof(Stack));
    st->capacity = capacity;
    st->top = -1;
    st->data = (Node**)malloc(sizeof(Node*) * capacity);
    return st;
}

void push(Stack* st, Node* node) {
    if (st->top == st->capacity - 1) {
        st->capacity *= 2;
        st->data = (Node**)realloc(st->data, sizeof(Node*) * st->capacity)
```

```
        st->capacity *= 2;
        st->data = (Node**)realloc(st->data, sizeof(Node*) * st->capacity);
    }
    st->data[++st->top] = node;
}

Node* pop(Stack* st) {
    if (st->top == -1) return NULL;
    return st->data[st->top--];
}

int is_stack_empty(Stack* st) {
    return st->top == -1;
}

void free_stack(Stack* st) {
    free(st->data);
    free(st);
}

void preorderIte(Node* root) {
    if (root == NULL)
        return;

    Stack* st = create_stack(16);
    push(st, root);

    while (!is_stack_empty(st)) {
        Node* curr = pop(st);

        printf("%d ", curr->val);

        if (curr->right)
            push(st, curr->right);
        if (curr->left)
            push(st, curr->left);
    }

    free_stack(st);
}

// --- Queue Utilities & Levelorder / Tree Visualization ---

Queue* create_queue(int capacity) {
    Queue* q = (Queue*)malloc(sizeof(Queue));
    q->capacity = capacity;
    q->front = 0;
    q->rear = 0;
    q->data = (Node**)malloc(sizeof(Node*) * capacity);
    return q;
}

void enqueue(Queue* q, Node* node) {
    if (q->rear == q->capacity) {
```

```c

int main() {
    Node* root = NULL;

    // Building the binary search tree
    add(10, &root);
    add(5, &root);
    add(15, &root);
    add(3, &root);
    add(7, &root);
    add(12, &root);
    add(18, &root);

    printf("Visualized Tree Layout:\n");
    printTree(root);

    printf("Inorder Traversal: ");
    inorder(root);
    printf("\n");

    printf("Preorder Recursive: ");
    preorderRec(root);
    printf("\n");

    printf("Preorder Iterative: ");
    preorderIte(root);
    printf("\n");

    printf("Postorder Traversal: ");
    postorder(root);
    printf("\n");

    printf("Levelorder Traversal: ");
    levelorder(root);
    printf("\n");

    printf("\nTree Statistics:\n");
    printf("Height: %d\n", height(root));
    printf("Min Value: %d\n", mini(root));
    printf("Max Value: %d\n", maxi(root));

    int target = 7;
    Node* result = search(root, target);
    if (result) {
        printf("Search %d: Found\n", target);
    } else {
        printf("Search %d: Not Found\n", target);
    }

    // Clean up allocated tree nodes
    free_tree(root);

    return 0;
}
```

```c
void enqueue(Queue* q, Node* node) {
    if (q->rear == q->capacity) {
        q->capacity *= 2;
        q->data = (Node**)realloc(q->data, sizeof(Node*) * q->capacity);
    }
    q->data[q->rear++] = node;
}

Node* dequeue(Queue* q) {
    if (q->front == q->rear) return NULL;
    return q->data[q->front++];
}

int queue_size(Queue* q) {
    return q->rear - q->front;
}

int is_queue_empty(Queue* q) {
    return q->front == q->rear;
}

void free_queue(Queue* q) {
    free(q->data);
    free(q);
}

void levelorder(Node* root) {
    if (root == NULL)
        return;

    Queue* q = create_queue(16);
    enqueue(q, root);

    while (!is_queue_empty(q)) {
        Node* curr = dequeue(q);

        printf("%d ", curr->val);

        if (curr->left)
            enqueue(q, curr->left);
        if (curr->right)
            enqueue(q, curr->right);
    }

    free_queue(q);
}

void printTree(Node* root) {
    if (root == NULL)
        return;

    int h = height(root);
    Queue* q = create_queue(32);
    enqueue(q, root);
```

```c
void printTree(Node* root) {
    if (root == NULL)
        return;

    int h = height(root);
    Queue* q = create_queue(32);
    enqueue(q, root);

    int level = 0;

    while (!is_queue_empty(q)) {
        int nodes = queue_size(q);
        int firstSpace = (1 << (h - level)) - 1;
        int betweenSpace = (1 << (h - level + 1)) - 1;

        for (int i = 0; i < firstSpace; i++)
            printf(" ");

        while (nodes--) {
            Node* curr = dequeue(q);

            if (curr) {
                printf("%d", curr->val);
                enqueue(q, curr->left);
                enqueue(q, curr->right);
            } else {
                printf(" ");
                enqueue(q, NULL);
                enqueue(q, NULL);
            }

            for (int i = 0; i < betweenSpace; i++)
                printf(" ");
        }

        printf("\n\n");

        level++;
        if (level == h)
            break;
    }

    free_queue(q);
}

// --- Main Executable ---

int main() {
    Node* root = NULL;

    // Building the binary search tree
    add(10, &root);
    add(5, &root);
    add(15, &root);
```

Part B] Binary Search Tree (strings)

Aim:- To design, implement C program for BST storing no string values via linked list. The Algorithm handles string search and insertion based on lexicographical order.

Algorithm:

i] Search and Insertion (root, str)
- if (root == Null) root = new Node(str);
- compare str with curr node via strcmp();
- if (strcmp < 0) go left node
  strcmp > 0 go right Node
  else return (ignore duplicates).

ii] Level order printing (root)
- enque root Node.
- while queue ≠ empty
  - count elements in queue.
  - dequeue those many nodes to print and enqueue their children if exists
  - print new line with level number.

iii] Preorder (root)
- if (node == null) return
- print curr node string
- call preorder (curr->left) if exists.
- call preorder (curr->right) if exists.

(iv) preorder (root) (Iterative)
- if root == null return
- push root in stack.
- while (stack ≠ empty)
  - pop stack top and print node string
  - push (curr->right) if exists
  - push (curr->left) if exists.

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

// --- Structure Definitions ---

typedef struct Node {
    char* val;
    struct Node* left;
    struct Node* right;
} Node;

typedef struct Stack {
    Node** data;
    int top;
    int capacity;
} Stack;

typedef struct Queue {
    Node** data;
    int front;
    int rear;
    int capacity;
} Queue;

// --- Node Helper Functions ---

Node* create_node(const char* val) {
    Node* new_node = (Node*)malloc(sizeof(Node));
    if (!new_node) return NULL;

    // Allocate memory for the string length + 1 for null terminator
    new_node->val = (char*)malloc(strlen(val) + 1);
    if (!new_node->val) {
        free(new_node);
        return NULL;
    }

    strcpy(new_node->val, val);
    new_node->left = NULL;
    new_node->right = NULL;
    return new_node;
}
void free_tree(Node* root) {
    if (root == NULL) return;
    free_tree(root->left);
    free_tree(root->right);
    free(root->val); // Free dynamically allocated string memory
    free(root);
}

// --- Tree Operations ---

void add(const char* data, Node** root) {
```

```
53
54 void add(const char* data, Node** root) {
55     if (*root == NULL) {
56         *root = create_node(data);
57         return;
58     }
59
60     Node* temp = *root;
61     while (1) {
62         int cmp = strcmp(data, temp->val);
63         if (cmp < 0) {
64             if (temp->left == NULL) {
65                 temp->left = create_node(data);
66                 return;
67             }
68             temp = temp->left;
69         } else if (cmp > 0) {
70             if (temp->right == NULL) {
71                 temp->right = create_node(data);
72                 return;
73             }
74             temp = temp->right;
75         } else {
76             return; // Duplicate string value
77         }
78     }
79 }
80
81 Node* search(Node* root, const char* data) {
82     Node* temp = root;
83     while (temp != NULL) {
84         int cmp = strcmp(data, temp->val);
85         if (cmp < 0)
86             temp = temp->left;
87         else if (cmp > 0)
88             temp = temp->right;
89         else
90             return temp;
91     }
92     return NULL;
93 }
94
95 const char* maxi(Node* root) {
96     if (root == NULL)
97         return NULL;
98
99     while (root->right != NULL)
100        root = root->right;
101
102    return root->val;
103 }
104
105 const char* mini(Node* root) {
106    if (root == NULL)
```

```c
const char* mini(Node* root) {
    if (root == NULL)
        return NULL;

    while (root->left != NULL)
        root = root->left;

    return root->val;
}

static inline int max_int(int a, int b) {
    return (a > b) ? a : b;
}

int height(Node* root) {
    if (root == NULL)
        return 0;

    return 1 + max_int(height(root->left), height(root->right));
}

// --- Recursive Traversals ---

void inorder(Node* root) {
    if (root == NULL)
        return;

    inorder(root->left);
    printf("%s ", root->val);
    inorder(root->right);
}

void preorderRec(Node* root) {
    if (root == NULL)
        return;

    printf("%s ", root->val);
    preorderRec(root->left);
    preorderRec(root->right);
}

void postorder(Node* root) {
    if (root == NULL)
        return;

    postorder(root->left);
    postorder(root->right);
    printf("%s ", root->val);
}

// --- Stack Utilities & Iterative Preorder Traversal ---

Stack* create_stack(int capacity) {
    Stack* st = (Stack*)malloc(sizeof(Stack));
```

```
Stack* create_stack(int capacity) {
    Stack* st = (Stack*)malloc(sizeof(Stack));
    st->capacity = capacity;
    st->top = -1;
    st->data = (Node**)malloc(sizeof(Node*) * capacity);
    return st;
}

void push(Stack* st, Node* node) {
    if (st->top == st->capacity - 1) {
        st->capacity *= 2;
        st->data = (Node**)realloc(st->data, sizeof(Node*) * st->capacity)
    }
    st->data[++st->top] = node;
}

Node* pop(Stack* st) {
    if (st->top == -1) return NULL;
    return st->data[st->top--];
}

int is_stack_empty(Stack* st) {
    return st->top == -1;
}

void free_stack(Stack* st) {
    free(st->data);
    free(st);
}

void preorderIte(Node* root) {
    if (root == NULL)
        return;

    Stack* st = create_stack(16);
    push(st, root);

    while (!is_stack_empty(st)) {
        Node* curr = pop(st);

        printf("%s ", curr->val);

        if (curr->right)
            push(st, curr->right);
        if (curr->left)
            push(st, curr->left);
    }

    free_stack(st);
}

// --- Queue Utilities & Levelorder / Tree Visualization ---

Queue* create_queue(int capacity) {
```

```c
Queue* create_queue(int capacity) {
    Queue* q = (Queue*)malloc(sizeof(Queue));
    q->capacity = capacity;
    q->front = 0;
    q->rear = 0;
    q->data = (Node**)malloc(sizeof(Node*) * capacity);
    return q;
}

void enqueue(Queue* q, Node* node) {
    if (q->rear == q->capacity) {
        q->capacity *= 2;
        q->data = (Node**)realloc(q->data, sizeof(Node*) * q->capacity);
    }
    q->data[q->rear++] = node;
}

Node* dequeue(Queue* q) {
    if (q->front == q->rear) return NULL;
    return q->data[q->front++];
}

int queue_size(Queue* q) {
    return q->rear - q->front;
}

int is_queue_empty(Queue* q) {
    return q->front == q->rear;
}

void free_queue(Queue* q) {
    free(q->data);
    free(q);
}

void levelorder(Node* root) {
    if (root == NULL)
        return;

    Queue* q = create_queue(16);
    enqueue(q, root);

    while (!is_queue_empty(q)) {
        Node* curr = dequeue(q);

        printf("%s ", curr->val);

        if (curr->left)
            enqueue(q, curr->left);
        if (curr->right)
            enqueue(q, curr->right);
    }

    free_queue(q);
```

```
}

void printTree(Node* root) {
    if (root == NULL)
        return;

    int h = height(root);
    Queue* q = create_queue(32);
    enqueue(q, root);

    int level = 0;

    while (!is_queue_empty(q)) {
        int nodes = queue_size(q);
        int firstSpace = (1 << (h - level)) - 1;
        int betweenSpace = (1 << (h - level + 1)) - 1;

        for (int i = 0; i < firstSpace; i++)
            printf(" ");

        while (nodes--) {
            Node* curr = dequeue(q);

            if (curr) {
                printf("%s", curr->val);
                enqueue(q, curr->left);
                enqueue(q, curr->right);
            } else {
                printf(" ");
                enqueue(q, NULL);
                enqueue(q, NULL);
            }

            for (int i = 0; i < betweenSpace; i++)
                printf(" ");
        }

        printf("\n\n");

        level++;
        if (level == h)
            break;
    }

    free_queue(q);
}

// --- Main Executable ---

int main() {
    Node* root = NULL;

    // Building the binary search tree with strings (Lexicographical order
    add("Mango", &root);
```

```c
int main() {
    Node* root = NULL;

    // Building the binary search tree with strings (Lexicographical order
    add("Mango", &root);
    add("Apple", &root);
    add("Peach", &root);
    add("Banana", &root);
    add("Grape", &root);
    add("Orange", &root);
    add("Plum", &root);

    printf("Visualized Tree Layout:\n");
    printTree(root);

    printf("Inorder Traversal (Alphabetical Order): ");
    inorder(root);
    printf("\n");

    printf("Preorder Recursive: ");
    preorderRec(root);
    printf("\n");

    printf("Preorder Iterative: ");
    preorderIte(root);
    printf("\n");

    printf("Postorder Traversal: ");
    postorder(root);
    printf("\n");

    printf("Levelorder Traversal: ");
    levelorder(root);
    printf("\n");

    printf("\nTree Statistics:\n");
    printf("Height: %d\n", height(root));
    printf("Min Value (Lexicographically First): %s\n", mini(root));
    printf("Max Value (Lexicographically Last): %s\n", maxi(root));

    const char* target = "Grape";
    Node* result = search(root, target);
    if (result) {
        printf("Search \"%s\": Found\n", target);
    } else {
        printf("Search \"%s\": Not Found\n", target);
    }

    // Clean up allocated tree nodes and string buffers
    free_tree(root);

    return 0;
}
```